

## MODULE 35

# Frontend to API Integration

Building seamless connections between your React UI and the Spring Boot backend -- Axios, the Fetch API, SOAP consumption, XML parsing, and JSON mapping.





# MODULE 35 OVERVIEW

*Connect the React frontend to the Spring Boot CRM API -- typed HTTP clients, request states, error handling, and secure integration.*

Theory: 1 Hour · Lab: 2 Hours · Total: 3 Hours -- confirmed against the official lesson-plan spreadsheet (Module 35 row).

## DURATION & LAB



### 1 Hour Theory

Client-server communication, the request-response lifecycle, Axios, API responses, error handling, and loading states.



### 2 Hours Lab

Lab 35: wire the React CRM SPA to a real Spring Boot customer API with a typed fetch client.



### Scenario

Freeze the SPA <-> API contract: typed fetch, AbortController cleanup, explicit request states, CORS, and failure-class tests.

## MODULE ARC



### Master Axios & Fetch

Send GET, POST, PUT, and DELETE requests and handle their responses correctly.



### Handle Failure Gracefully

Loading, empty, error, and data states -- plus network, 4xx, and 5xx failure classes.



### Build the Lab

Connect lab35-crm to a running Spring Boot API with CORS, correlation IDs, and tests.

**KEY TAKEAWAY** Total time: 3 Hours (1 Hour Theory + 2 Hours hands-on Lab). By the end, you will connect a React frontend to a Spring Boot REST API with production-grade discipline.

# WHY API INTEGRATION MATTERS

*APIs connect your frontend applications to powerful backend services and data.*

## WHY API INTEGRATION MATTERS

- **Enables Separation of Concerns** — keeps frontend and backend independent, improving maintainability and scalability.
- **Improves Scalability** — APIs allow services to scale independently based on demand.
- **Enhances Reusability** — APIs expose functionality that can be reused across multiple clients and platforms.
- **Supports Multiple Clients** — the same API can power web apps, mobile apps, and third-party integrations.
- **Faster Development** — frontend teams can work in parallel with backend teams using well-defined APIs.
- **Strengthens Security** — centralized business logic and validation on the server reduce client-side vulnerabilities.

## THE ROLE OF APIS IN MODERN APPLICATIONS



## KEY TAKEAWAY

API integration is the bridge between your user interface and your business logic. It enables communication, data exchange, and delivers seamless experiences to your users.

**KEY TAKEAWAY** API integration is the bridge between your user interface and your business logic -- it enables communication, data exchange, and seamless user experiences.

# MODULE LEARNING OBJECTIVES

*By the end of this module, you will be able to connect a modern React application to a robust Spring Boot API and build dynamic, data-driven user experiences.*

- **1. Understand the Lifecycle** — explain client-server communication and the request-response lifecycle.
- **2. Use Axios** — send GET, POST, PUT, and DELETE requests to RESTful APIs.
- **3. Handle Responses** — handle API responses and manage data effectively in frontend applications.
- **4. Implement Error Handling** — implement error handling and manage loading states to improve user experience.
- **5. Integrate with Spring Boot** — integrate React applications with Spring Boot REST APIs using best practices.
- **6. Apply Security Best Practices** — apply security best practices when communicating between frontend and backend.

## MODULE FOCUS AREAS

Axios & Fetch

Response & Error Handling

Loading States

Spring Boot Integration

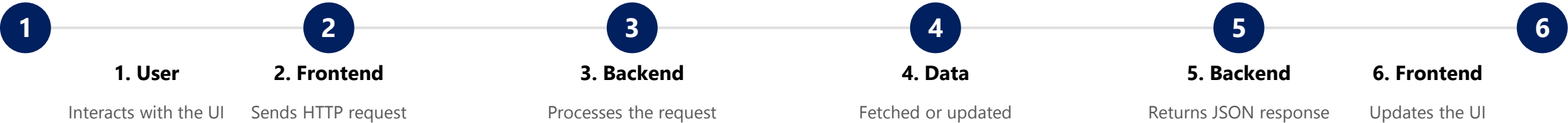
Frontend Security

**KEY TAKEAWAY** You will build practical skills to connect modern React applications with robust Spring Boot APIs, creating dynamic, data-driven user experiences.

# FRONTEND-BACKEND ARCHITECTURE

*A frontend-backend architecture separates the user interface from the server-side logic and data management.*

## HOW IT WORKS



## KEY BENEFITS

- **Separation of Concerns** — UI/UX and business logic are developed and maintained independently.
- **Scalability** — frontend and backend can scale independently based on demand.
- **Flexibility** — multiple frontends (web, mobile, desktop) can consume the same backend APIs.
- **Maintainability** — easier to update, test, and deploy components independently.
- **Better Team Collaboration** — frontend and backend teams work in parallel with clear contracts (APIs).

## ARCHITECTURE LAYERS

| Layer      | Responsibilities                                                                 |
|------------|----------------------------------------------------------------------------------|
| Frontend   | Components, State Management, Routing, UI/UX (React)                             |
| Backend    | Controllers, Services, Repositories, Security, Validation (Spring Boot)          |
| Data Layer | Database (PostgreSQL / MySQL), other services (email, storage, third-party APIs) |

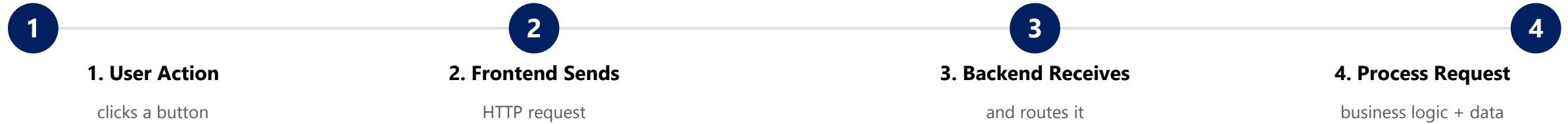
**KEY TAKEAWAY** This architecture promotes modularity, reusability, and scalability while delivering a seamless experience to users.



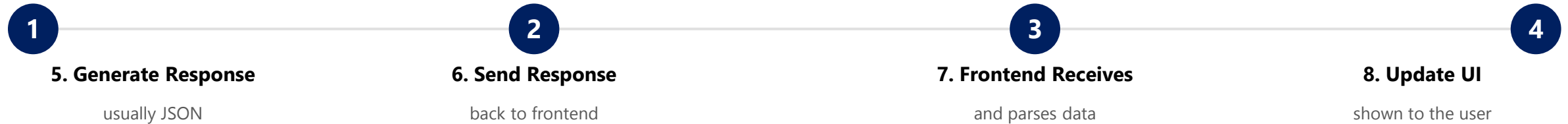
# REQUEST-RESPONSE LIFECYCLE

*Every interaction between the frontend and backend follows a structured request-response lifecycle.*

## REQUEST FLOW



## RESPONSE FLOW



## KEY POINTS



## EXAMPLE -- "VIEW USERS"

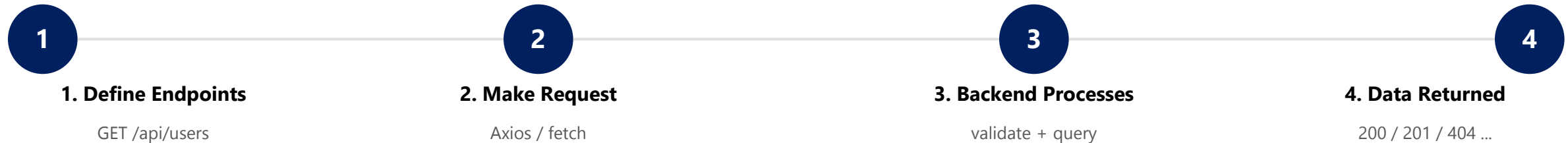
- Click → GET /api/users → backend queries the database → JSON response → Axios receives it → React state updates → list is displayed.

**KEY TAKEAWAY** Understanding this lifecycle is key to building reliable, efficient, and user-friendly applications.

# API INTEGRATION WORKFLOW (1 OF 2)

*A structured workflow ensures seamless communication between the frontend application and backend APIs -- steps 1-4: from defining the endpoint to getting data back.*

## STEPS 1-4: FROM ENDPOINT TO RESPONSE



## REAL-WORLD EXAMPLE

- User clicks "View Users" → React sends GET /api/users.
- Spring Boot fetches users from the database and returns JSON.
- Axios receives the response; data is set into React state.
- The user list is displayed on the screen.

**KEY TAKEAWAY** The first half of the workflow is about getting a well-defined request out the door and a response back -- define the endpoint, make the request, let the backend process it, and get data back with a meaningful status code.



# API INTEGRATION WORKFLOW (2 OF 2)

*Steps 5-7: from receiving the response to a seamless UI update -- plus the best practices that keep the whole workflow reliable.*

## STEPS 5-7: FROM RESPONSE TO UPDATED UI



## BEST PRACTICES

Meaningful endpoint names

Use HTTPS

Handle loading states

Handle errors gracefully

## KEY TAKEAWAY

A well-defined API integration workflow ensures efficient communication, better error handling, and a smooth user experience.

**KEY TAKEAWAY** Keep API contracts consistent and well-documented -- from defining endpoints to updating the UI, every step should be predictable and repeatable.

# INTRODUCTION TO AXIOS

*Axios is a promise-based HTTP client for the browser and Node.js. It makes it easy to send asynchronous HTTP requests to RESTful APIs and handle responses.*

## KEY FEATURES OF AXIOS

- **Promise Based** — built on top of Promises, making async code easier to write and manage.
- **Easy to Use** — simple and intuitive API for making HTTP requests.
- **Interceptors** — intercept requests or responses before they are handled.
- **Error Handling** — automatically handles errors and provides meaningful messages.
- **Browser & Node.js Support** — works seamlessly in both browsers and Node.js environments.

## INSTALLATION

```
npm install axios
```

## BASIC EXAMPLE

```
import axios from 'axios';

// Send a GET request
axios.get('https://api.example.com/users')
  .then(response => {
    console.log('Data:', response.data);
  })
  .catch(error => {
    console.error('Error:', error);
  });
```

## WHY USE AXIOS?

Simplifies requests

Handles JSON automatically

Interceptors

Better error handling

**KEY TAKEAWAY** Axios provides a clean, simple, and powerful way to interact with APIs, making it an essential tool for frontend development.

# SENDING GET REQUESTS

*GET requests are used to retrieve data from the server. They should not modify any data on the server.*

## SYNTAX

```
axios.get(url[, config])
```

## EXAMPLE -- GET WITH QUERY PARAMETERS

```
axios.get('https://api.example.com/users', {
  params: { page: 1, size: 10 }
})
.then(response => console.log(response.data))
.catch(error => console.error('Error:', error));
```

## COMMON STATUS CODES

200 OK

400 Bad Request

401 Unauthorized

404 Not Found

500 Server Error

## RESPONSE STRUCTURE

| Property         | Meaning                                    |
|------------------|--------------------------------------------|
| data             | The response data returned from the server |
| status           | HTTP status code (e.g., 200)               |
| statusText       | Status message (e.g., OK)                  |
| headers          | Response headers                           |
| config / request | Original request config / request instance |

## BEST PRACTICES

- Use meaningful, RESTful endpoint names.
- Use query parameters for filtering, sorting, and pagination.
- Avoid exposing sensitive data in URLs.
- Show loading indicators while fetching data.

**KEY TAKEAWAY** Use `axios.get()` to retrieve data from the server. Handle the response and errors gracefully to build reliable applications.

# SENDING POST REQUESTS

*POST requests are used to send data to the server to create new resources.*

## SYNTAX

```
axios.post(url, data, config)
```

## EXAMPLE -- POST WITH HEADERS

```
const customer = {
  fullName: 'Amina Khan',
  email: 'amina@example.com',
  status: 'ACTIVE',
};
axios.post('https://api.example.com/customers', customer, {
  headers: { 'Content-Type': 'application/json' },
})
.then(response => console.log(response.data))
.catch(error => console.error('Error:', error));
```

## COMMON STATUS CODES

201 Created

400 Bad Request

409 Conflict

500 Server Error

## EXAMPLE RESPONSE (201 CREATED)

```
{
  "data": {
    "id": "CUS-1001",
    "fullName": "Amina Khan",
    "email": "amina@example.com",
    "status": "ACTIVE"
  },
  "status": 201,
  "statusText": "Created"
}
```

## BEST PRACTICES

- Validate data on both frontend and backend.
- Always send data in JSON format.
- Use HTTPS and authentication for secure communication.
- Disable the submit control while the request is in flight.

**KEY TAKEAWAY** Use `axios.post()` to send data to the server and create new resources. Always handle responses and errors gracefully -- and guard against duplicate submits.

# SENDING PUT REQUESTS

*PUT requests are used to update an existing resource on the server. They replace the entire resource with the new data.*

## SYNTAX

```
axios.put(url, data, config)
```

## EXAMPLE -- UPDATE A CUSTOMER

```
const draft = {
  fullName: 'Ravi Singh',
  email: 'ravi@example.com',
  status: 'ACTIVE',
};
axios.put(
  `https://api.example.com/customers/${id}`,
  draft
)
.then(response => console.log(response.data));
```

## RESPONSE STRUCTURE

| Property | Meaning                                       |
|----------|-----------------------------------------------|
| data     | The updated resource returned from the server |
| status   | 200 (OK) or 204 (No Content)                  |

## BEST PRACTICES

- Use PUT when you want to replace the entire resource.
- Ensure the resource exists before updating (encodeURIComponent the id).
- Validate data on both frontend and backend.
- Prefer the server-returned record over an optimistic-only update.

## COMMON STATUS CODES

200 OK

204 No Content

400 Bad Request

404 Not Found

**KEY TAKEAWAY** Use axios.put() to update existing resources on the server. It replaces the entire resource with the new data.

# SENDING DELETE REQUESTS

*DELETE requests are used to remove a resource from the server. Once deleted, the resource is no longer available.*

## SYNTAX

```
axios.delete(url, config)
```

## EXAMPLE -- DELETE A RESOURCE

```
axios.delete('https://api.example.com/customers/CUS-1002')
  .then(() => {
    console.log('Customer deleted successfully');
  })
  .catch(error => console.error('Error:', error));
```

## BEST PRACTICES

- Confirm the resource exists before attempting to delete it.
- Use appropriate authentication and authorization.
- Use HTTPS to protect sensitive operations.
- Show a confirmation prompt in the UI before deleting.
- Log delete operations for auditing purposes.

## RESPONSE STRUCTURE

| Response       | Meaning                                                   |
|----------------|-----------------------------------------------------------|
| 200 OK         | Deleted; response body may contain a confirmation message |
| 204 No Content | Deleted; no body returned                                 |

## COMMON STATUS CODES

200 OK

204 No Content

401 Unauthorized

404 Not Found

**KEY TAKEAWAY** Use `axios.delete()` to remove resources from the server. Always handle responses and errors carefully and confirm actions to prevent accidental data loss.

# HANDLING API RESPONSES

*After sending a request, the API returns a response. Properly handling the response is crucial for a great user experience and robust applications.*

## RESPONSE FLOW



### 4xx Client Error (404)

```
{
  "status": 404,
  "statusText": "Not Found",
  "error": "RESOURCE_NOT_FOUND",
  "message": "User not found"
}
```

### 5xx Server Error (500)

```
{
  "status": 500,
  "statusText": "Internal Server Error",
  "error": "INTERNAL_SERVER_ERROR",
  "message": "Something went wrong"
}
```

## COMMON HTTP STATUS CODES

200 OK

201 Created

400 Bad Request

404 Not Found

500 Server Error

## HANDLE RESPONSES IN REACT (EXAMPLE)

```
const fetchUsers = async () => {
  try {
    const response = await axios.get('/api/users');
    console.log('Data:', response.data);
    setUsers(response.data);
  } catch (error) {
    if (error.response) {
      // Server responded with a status outside 2xx
      console.error('Status:', error.response.status);
      setError(error.response.data.message);
    } else if (error.request) {
      // Request made, no response received
      console.error('No response:', error.request);
      setError('Cannot reach the server.');
```

**KEY TAKEAWAY** Always handle API responses by checking status codes, processing data, and managing errors effectively to build reliable user experiences.



# ERROR HANDLING IN FRONTEND APPLICATIONS

*Errors can happen anywhere: network issues, server errors, invalid responses, or unexpected bugs. Good error handling improves user experience and reliability.*

## TYPES OF ERRORS

- **Network Errors** — no internet, request timeout, DNS failure.
- **Server Errors** — API returns 4xx (client) or 5xx (server) status codes.
- **Client/Validation Errors** — invalid input, missing fields, business-rule violations.
- **Unexpected Errors** — bugs in code, undefined states, parsing issues.

## SHOWING USER-FRIENDLY MESSAGES

- **Be Clear** — use simple language and avoid technical jargon.
- **Be Helpful** — suggest what the user can do next (e.g., Retry).
- **Be Contextual** — show messages near the relevant field or section.
- **Be Dismissible** — allow the user to close or retry the message.

## HANDLING API ERRORS (BEST PRACTICES)

- Always handle errors in every API call.
- Provide a retry option for recoverable errors.
- Show loading state and disable actions while loading.
- Log errors for debugging (e.g., Sentry, LogRocket).
- Never expose sensitive information in error messages.

## EXAMPLE UI

| Message shown to the user                            |
|------------------------------------------------------|
| ⚠ <b>Failed to load customers. Please try again.</b> |
| ⚠ <b>Please check your input and try again.</b>      |
| ⚠ <b>No internet connection. Check your network.</b> |

**KEY TAKEAWAY** Handling errors gracefully makes your application trustworthy, user-friendly, and production-ready.

# TRY IT YOURSELF -- EXERCISE 1: ERROR UX COPY

*Reinforces: user-facing failure messages for 404, network, and 400 CRM API errors -- a documentation exercise.*

**DRAFT USER-FACING MESSAGES (RECORD IN `notes/error-ux.md`)**

| Step                        | What To Do                                                                                                              |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 1 -- 404 Message            | Draft the message shown when GET /api/customers/CUS-9999 returns 404: the customer was not found.                       |
| 2 -- Network Message        | Draft the message shown when the API is unreachable -- no response, a timeout, or a CORS rejection.                     |
| 3 -- 400 Message            | Draft the message shown when a create/update request fails name validation with a 400 Bad Request.                      |
| 4 -- Logging vs UX Boundary | Write the rule: the dev console may log the X-Correlation-Id and status code -- the user only ever sees plain language. |

**PASS CRITERIA**

- **404 / network / 400 messages** -- all three are drafted in plain language, each naming what happened without jargon.
- **Correlation stays in logs** -- X-Correlation-Id and raw status codes are noted as dev-console/log detail, never shown to the user.
- **No stack traces in UI copy** -- none of the three messages leak a stack trace, SQL error, or internal exception name.

**KEY TAKEAWAY** TRY IT NOW -- Complete Exercise 1 before continuing: `exercises/exercise-01-error-ux.md` (workspace: `examples/module-35-exercises`).

# LOADING STATES AND USER EXPERIENCE

Loading states inform users that something is happening. They reduce uncertainty, prevent duplicate actions, and create a smooth, professional experience.

## WHY LOADING STATES MATTER

- **Reduce Uncertainty** — users know the app is working and their action was received.
- **Prevent Duplicate Actions** — disable buttons or show loaders to avoid multiple submissions.
- **Improve Perceived Performance** — users are more patient when kept informed.
- **Enhance Professionalism** — good UX builds trust and confidence in your application.

## EXAMPLE: USER LIST COMPONENT

| Loading                 | Success                 | Error                   |
|-------------------------|-------------------------|-------------------------|
| Skeleton rows / spinner | Customer cards rendered | ⚠ Failed to load. Retry |

## BEST PRACTICES

- Show loading immediately
- Disable inputs while loading
- Never hide errors

## IMPLEMENTING LOADING STATES (REACT + FETCH)

```
const [customers, setCustomers] = useState([]);
const [loading, setLoading] = useState(false);
const [error, setError] = useState(null);

const fetchCustomers = async () => {
  setLoading(true);
  setError(null);
  try {
    const data = await customersApi.list();
    setCustomers(data);
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false); // always hide the loader
  }
};
```

## COMMON LOADING UI PATTERNS

- Spinner
- Skeleton Screen
- Button Loader
- Progress Bar

**KEY TAKEAWAY** Well-designed loading states keep users informed, reduce frustration, and create a smoother, more reliable experience.

# TRY IT YOURSELF -- EXERCISE 2: FETCH FLOW

*Reinforces: the idle/loading/success/error state machine for listing customers, including abort and empty-state UX -- an architecture exercise.*

**SKETCH THE STATE MACHINE (RECORD IN `notes/fetch-flow.md`)**

```
idle -> loading -> success | error

useEffect(() => {
  const controller = new AbortController();
  setState('loading');
  customersApi
    .list({ signal: controller.signal })
    .then((data) => setState('success', data))
    .catch((err) => setState('error', err));
  return () => controller.abort(); // cleanup
}, []);
```

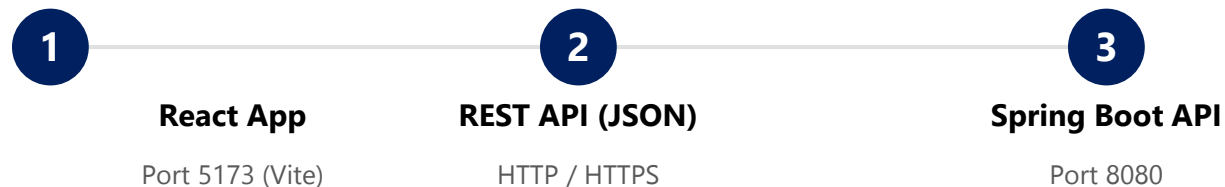
| # | What To Do                                                                                      |
|---|-------------------------------------------------------------------------------------------------|
| 1 | Name the four states for the customer list view: idle, loading, success, error.                 |
| 2 | Sequence: mount -> set loading -> fetch -> set data (Amina + Ravi) or set an error message.     |
| 3 | Note AbortController cleanup on unmount, so a late response can't setState after navigate-away. |
| 4 | Draft empty-state copy for when the API returns [] -- distinct from both loading and error.     |

**KEY TAKEAWAY** TRY IT NOW -- Complete Exercise 2 before continuing: `exercises/exercise-02-fetch-flow.md` (workspace: `examples/module-35-exercises`).

# CONNECTING REACT TO SPRING BOOT (1 OF 2)

*React (frontend) communicates with Spring Boot (backend) through REST APIs. This connection enables data exchange and builds full-stack applications.*

## APPLICATION ARCHITECTURE



## CONNECTION STEPS

- Start the Spring Boot application (usually on port 8080).
- Expose REST endpoints that return JSON.
- Start the React (Vite) development server (usually on port 5173).
- Configure the API base URL via a Vite environment variable.
- Use fetch or Axios in React to call REST endpoints.
- Spring Boot processes the request and returns JSON.
- React receives the response and updates the UI.

## CROSS-ORIGIN RESOURCE SHARING (CORS)

React and Spring Boot run on different origins (ports). CORS must be enabled on the backend so the browser allows the request.

### WebConfig.java

```
@Configuration
public class WebConfig implements WebMvcConfigurer {
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/api/**")
            .allowedOrigins("http://localhost:5173")
            .allowedMethods("GET", "POST", "PUT", "DELETE")
            .allowedHeaders("Content-Type", "X-Correlation-Id");
    }
}
```

## BEST PRACTICES

Env vars for base URL

Handle loading & error states

Allowlist specific origins only

**KEY TAKEAWAY** React communicates with Spring Boot using REST APIs over JSON. A restricted CORS allowlist and an environment-configured base URL keep the connection correct and secure.

# TRY IT YOURSELF -- EXERCISE 3: CORS AND HEADERS

*Reinforces: local CORS origins and the X-Correlation-Id header for the CRM SPA -- a documentation exercise.*

**DOCUMENT CORS + HEADERS (RECORD IN `notes/cors-and-headers.md`)**

| Step                  | What To Do                                                                                                                                                            |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 -- Origins          | Record the two local origins: UI at <code>http://localhost:5173</code> (Vite), API at <code>http://localhost:8080</code> (Spring Boot) -- adjust if your lab differs. |
| 2 -- Explain CORS     | Write one sentence: the browser blocks cross-origin fetch/XHR calls unless Spring's CORS config explicitly allows the UI's origin.                                    |
| 3 -- Plan the Header  | Note the header every fetch call carries: <code>X-Correlation-Id: lab-request-001</code> .                                                                            |
| 4 -- Secrets Boundary | Write the rule: never place DB passwords or other secrets in frontend env vars -- only the public API base URL belongs in <code>VITE_CRM_API_URL</code> .             |

**PREFLIGHT SYMPTOM (BROWSER CONSOLE) -- WHAT A MISSING ALLOWLIST LOOKS LIKE**

```
Access to fetch at 'http://localhost:8080/api/customers' from origin
'http://localhost:5173' has been blocked by CORS policy: No
'Access-Control-Allow-Origin' header is present on the requested resource.
```

**KEY TAKEAWAY** TRY IT NOW -- Complete Exercise 3 before continuing: `exercises/exercise-03-cors-and-headers.md` (workspace: `examples/module-35-exercises`).

# CONNECTING REACT TO SPRING BOOT (2 OF 2)

*A Spring Boot REST controller on one side, a typed fetch client on the other -- the same request/response contract, seen from both ends.*

## EXAMPLE: SPRING BOOT ENDPOINT

### CustomerController.java

```
@RestController
@RequestMapping("/api/customers")
public class CustomerController {

    @GetMapping
    public List<Customer> getAll() {
        return customerService.getAll();
    }

    @PostMapping
    public ResponseEntity<Customer> create(
        @Valid @RequestBody CustomerDraft draft) {
        Customer created = customerService.create(draft);
        return ResponseEntity.status(201).body(created);
    }
}
```

## EXAMPLE: REACT FETCH CLIENT

### api/customers.ts

```
const API_URL = import.meta.env.VITE_CRM_API_URL;

export async function request(path, init = {}) {
    const res = await fetch(`${API_URL}${path}`, {
        ...init,
        headers: {
            'Content-Type': 'application/json',
            'X-Correlation-Id': 'lab-request-001',
            ...init.headers,
        },
    });
    if (!res.ok) throw await ApiError.from(res);
    if (res.status === 204) return undefined;
    return res.json();
}
```

## COMMON HTTP METHODS

GET → Retrieve

POST → Create

PUT → Update

DELETE → Delete

**KEY TAKEAWAY** @RestController exposes JSON endpoints on the Spring side; one typed request() helper owns headers, error translation, and 204 handling on the React side.

# TRY IT YOURSELF -- EXERCISE 4: ENDPOINT MAP

*Reinforces: mapping CRM dashboard actions to Spring REST endpoints -- an analysis exercise.*

## UI ACTION -> ENDPOINT (RECORD IN notes/lab35-api.md)

| UI Action       | HTTP                        |
|-----------------|-----------------------------|
| List customers  | GET /api/customers          |
| Open Amina      | GET /api/customers/CUS-1001 |
| Open Ravi       | GET /api/customers/CUS-1002 |
| Create customer | POST /api/customers         |
| Update status   | PATCH /api/customers/{id}   |

## STEPS

| # | What To Do                                                                                        |
|---|---------------------------------------------------------------------------------------------------|
| 1 | Copy the reference table into notes/lab35-api.md; note any path differences from your Week 3 API. |
| 2 | Add the missing GET row for Ravi (CUS-1002).                                                      |
| 3 | List the five expected status codes: 200, 201, 400, 404, 500.                                     |
| 4 | Sketch the list-item JSON shape: customerId, name, status.                                        |

## LIST ITEM JSON SKETCH

```
{
  "customerId": "CUS-1001",
  "name": "Amina Khan",
  "status": "ACTIVE"
}
```

**KEY TAKEAWAY** TRY IT NOW -- Complete Exercise 4 before continuing: exercises/exercise-04-endpoint-map.md (workspace: examples/module-35-exercises).



# TRY IT YOURSELF -- EXERCISE 5: FILL FETCH TODOS (STARTER)

*Reinforces: completing a typed fetch helper for customers -- fill in every blank before it will compile.*

**FILL IN EVERY \_\_\_\_ -- customers.ts (STARTER, notes/lab35-todos.md)**

```
export type Customer = {
  customerId: string; name: string; status: string
};

export async function listCustomers(
  signal?: AbortSignal
): Promise<Customer[]> {
  const res = await fetch(____, {
    headers: { "X-Correlation-Id": "____" },
    signal,
  });
  if (!res.ok) throw new Error(`HTTP ${res.status}`);
  return (await res.json()) as ____;
}

export async function getCustomer(id: string) {
  const res = await fetch(`${____}/${id}`);
  // TODO: handle 404 for unknown id
  return (await res.json()) as Customer;
}
```

| # | What To Do                                                                        |
|---|-----------------------------------------------------------------------------------|
| 1 | Fill the fetch URL and header blanks: "/api/customers" and "lab-request-001".     |
| 2 | Fill the return-type blank with Customer[]; use the same base URL in getCustomer. |
| 3 | Add // TODO: on success setCustomers including Amina + Ravi fixtures from API.    |
| 4 | Add // TODO: map 400 body.detail to form error string.                            |

**KEY TAKEAWAY** TRY IT NOW -- This is a STARTER, not a solution: complete every blank in exercises/exercise-05-fill-fetch-todos.md, then verify it compiles/runs before continuing.

# API LAYER DESIGN (1 OF 2)

*A well-designed API layer acts as a bridge between frontend and backend, providing secure, reliable, and consistent access to business functionality.*

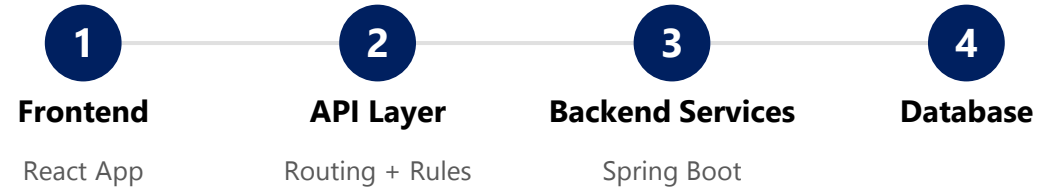
## WHAT IS AN API LAYER?

A dedicated layer that exposes application functionality through well-defined, secure, and standardized APIs -- the intermediary that handles requests, enforces rules, and returns properly structured responses.

## API LAYER RESPONSIBILITIES

- **Routing** — map incoming requests to the correct endpoint.
- **Validation** — validate path variables, query params, headers, and body.
- **Authentication & Authorization** — verify identity and permissions.
- **Business Logic Orchestration** — coordinate with services to process requests.
- **Response Formatting** — return consistent JSON responses and status codes.
- **Error Handling** — handle exceptions and return meaningful error responses.

## ROLE OF THE API LAYER



## DESIGN PRINCIPLES

- **Consistency** — consistent naming, data formats, and response structures across all endpoints.
- **Separation of Concerns** — keep routing, validation, business logic, and data access separated.
- **Stateless & Resource-Oriented** — each request is self-contained; APIs model business resources (nouns).
- **Security & Versioning** — authenticate every request; version APIs for backward compatibility.

**KEY TAKEAWAY** A well-designed API layer ensures reliable communication, security, and scalability -- focus on consistency, security, and a great developer experience.

# API LAYER DESIGN (2 OF 2)

*REST API design structure, a worked endpoint example, best practices, and common pitfalls to avoid.*

## REST API DESIGN STRUCTURE

| Layer      | Responsibility                                                       |
|------------|----------------------------------------------------------------------|
| Controller | Defines endpoints; handles HTTP requests/responses; validates input. |
| Service    | Implements business logic; coordinates operations and transactions.  |
| Repository | Data access via JPA; maps entities to persistent storage.            |

## EXAMPLE ENDPOINT

### GET /api/v1/customers/123

```
curl -X GET "http://localhost:8080/api/v1/customers/123" \
-H "X-Correlation-Id: lab-request-001"

// 200 OK
{ "data": { "id": 123, "fullName": "Amina Khan" },
  "status": 200, "message": "Customer retrieved" }
```

## BEST PRACTICES

Resource-based URIs, plural nouns

Correct HTTP methods

Meaningful status codes

Use DTOs to control exposure

Document with OpenAPI / Swagger

## COMMON PITFALLS

- Inconsistent URI naming conventions.
- Mixing business logic into controllers.
- Poor error handling and unclear responses.
- Exposing internal implementation details.
- Skipping API versioning.

**KEY TAKEAWAY** A well-designed API layer with clear contracts, consistent patterns, and robust error handling ensures a secure, scalable, and maintainable frontend-backend integration.

# FRONTEND SECURITY CONSIDERATIONS

*Security in frontend applications is critical to protect users, data, and your backend services. Remember: never trust the client -- always validate and enforce security on the server.*

## TOP SECURITY RISKS IN FRONTEND

- **Cross-Site Scripting (XSS)** — attackers inject malicious scripts to steal data or perform actions.
- **Cross-Site Request Forgery (CSRF)** — forces users to execute unwanted actions.
- **Sensitive Data Exposure** — exposing tokens, keys, or PII in the client.
- **Insecure Storage** — storing tokens or sensitive data in LocalStorage improperly.
- **Insecure API Calls** — calling APIs without authentication or over HTTP.
- **CORS Misconfiguration** — overly permissive CORS policies (e.g., `allowedOrigins("*")`).

## SECURE FRONTEND PRACTICES

- **Sanitize User Input** — encode or sanitize input before rendering.
- **Use HTTPS** — always use HTTPS for API calls and static assets.
- **Store Tokens Securely** — prefer HttpOnly cookies over localStorage for tokens.
- **Implement CSP** — restrict sources for scripts, styles, images, and frames.
- **Validate & Authorize on the Server** — never trust data from the client.
- **Handle Errors Safely** — show user-friendly messages; never expose internal details.

## SECURITY CHECKLIST

All API calls use HTTPS

Tokens stored securely

Input validated & sanitized

Dependencies up to date

CORS configured correctly

**KEY TAKEAWAY** Frontend security is about reducing attack surface, protecting user data, and using secure practices at every step -- always combine frontend safeguards with strong backend security.

# TRY IT YOURSELF -- EXERCISE 6: LAB 35 READINESS CHECKLIST

Capstone pre-lab check -- confirm the API contract and scope boundaries before opening Lab 35.

**READINESS CHECKLIST (RECORD IN `notes/lab35-readiness.md`)**

| Check             | What To Confirm                                                                                                         |
|-------------------|-------------------------------------------------------------------------------------------------------------------------|
| Dependency        | Lab 35 needs a running Spring API in the real lab -- confirm this pre-lab does NOT start Spring Boot or Vite.           |
| Evidence Preview  | Describe the Network-panel evidence you will capture in the lab: a GET list call plus the Amina (CUS-1001) detail call. |
| Security Boundary | Write one line: Authorization/bearer-token headers wait for Lab 36 -- do not implement JWT login UI yet.                |
| Pass Mark         | State the pre-lab pass mark: Exercise 4's fetch TODOs are filled and verified before Lab 35 begins.                     |

**SETUP (POWERSHELL) -- FROM EXERCISES-INDEX.MD**

```
cd $env:USERPROFILE\java-bootcamp
New-Item -ItemType Directory -Force -Path examples\module-35-exercises | Out-Null
cd examples\module-35-exercises
java -version
```

**KEY TAKEAWAY** TRY IT NOW -- Complete Exercise 6 before opening Lab 35: `exercises/exercise-06-lab35-readiness.md`.

# LAB OVERVIEW -- FRONTEND TO API INTEGRATION

## *Lab 35: Integrating React with the Spring CRM API -- lab35-crm*

### BUSINESS SCENARIO

- The CRM React client must talk to Spring Boot over HTTPS/JSON in production; Spring persists to PostgreSQL and may bridge SOAP internally, but the browser speaks JSON only.
- Leadership freezes the gate: no merge of SPA ↔ API integration without typed fetch, AbortController cleanup, explicit request states, a CORS allowlist, and tests for 2xx / 4xx / 5xx / network / abort.
- You own that gate for listing Amina (CUS-1001) and Ravi (CUS-1002), creating a draft, mapping email 400s, and rejecting https://evil.example.

### LEARNING OBJECTIVES

- Document the REST (or SOAP-bridge) customer contract with curl evidence.
- Configure a public API base URL via Vite env (no secrets in VITE\_\*).
- Write a typed fetch client and a normalized ApiError.
- Render loading, success, empty, and error states distinctly.
- Cancel obsolete requests with AbortController.
- Map backend validation errors to labeled form fields; configure restricted CORS.

### WHAT YOU BUILD

- lab35-crm/crm-ui: typed ApiError + http.request + customersApi; abortable list load with distinct loading/empty/data/error UI; create/update with X-Correlation-Id: lab-request-001 and a duplicate-submit guard; backend 400 email-field mapping; Spring CORS allowlist for localhost:5173; mock tests for 200/201/400/500/network/abort; evil-Origin curl evidence.

**KEY TAKEAWAY** Fixtures CUS-1001 (Amina Khan, ACTIVE) and CUS-1002 (Ravi Singh, PROSPECT) anchor every example; every mutating call carries X-Correlation-Id: lab-request-001; secrets never go in VITE\_\*.

# LAB TASKS AND DELIVERABLES

*lab35-crm -- implementation steps, deliverables, and the evaluation rubric.*

| # | Implementation Step                                                                                         |
|---|-------------------------------------------------------------------------------------------------------------|
| 1 | Inspect the API contract -- curl list/create; confirm CUS-1001/CUS-1002 JSON shapes.                        |
| 2 | Configure VITE_CRM_API_URL via .env.example (no secrets in VITE_*).                                         |
| 3 | Create the normalized ApiError type (status, code, message, fieldErrors).                                   |
| 4 | Build the http.request fetch boundary -- headers, 204 guard, non-OK → ApiError.                             |
| 5 | Create typed customersApi (list / create / update) on top of http.request.                                  |
| 6 | Cancel obsolete loads with AbortController in the list-load effect.                                         |
| 7 | Render distinct loading / empty / data / error states; map 400 field errors to the form.                    |
| 8 | Prevent duplicate POSTs (saving flag); restrict Spring CORS; test all response classes + probe evil Origin. |

## EXPECTED DELIVERABLES

- Typed ApiError / http / customersApi with abortable loads.
- Distinct loading / empty / data / error UX.
- Create/update with correlation header + duplicate-submit guard.
- Backend 400 field mapping.
- Spring CORS allowlist + evil-Origin evidence.
- Response-class tests + green build (twice).
- Integration notes + screenshots.
- README runbook (Spring + Vite); no secrets committed.

## RUBRIC (100 MARKS)

- Environment 10 · Core Impl 30 · Integration/Config 15 · Failure Handling 15 · Verification 10 · Security/Prod Awareness 10 · Docs 10

**KEY TAKEAWAY** Swallowing errors into empty lists loses failure-handling marks; leaving allowedOrigins("\*") in code is an honor violation -- prove abort, 400 mapping, and evil-Origin denial all work.

# MODULE 35 SUMMARY

*In this module, you connected a React frontend to a Spring Boot REST API, handled data and errors gracefully, and delivered a smooth user experience.*

## KEY CONCEPTS COVERED



### Axios & the 4 HTTP Verbs

GET, POST, PUT, and DELETE requests to RESTful Spring Boot APIs, and their response shapes.



### Request-Response Lifecycle

Client-server communication, the eight-step request/response flow, and status codes.



### Responses & Error Handling

Parsing response objects and handling network, server, and validation failures gracefully.



### Loading States

Distinct loading, empty, data, and error UX that keeps users informed and confident.



### React + Spring Boot Integration

CORS, environment configuration, and a well-designed API layer connecting the two.



### Frontend Security

Input sanitization, secure token storage, HTTPS, CSP, and server-side validation.

## MODULE FLOW RECAP

1

Axios / Fetch

2

4 HTTP Verbs

3

Handle Response

4

Loading & Error UX

5

CORS & Security

6

Lab: lab35-crm

**KEY TAKEAWAY** Frontend-to-API integration is the seam where your UI meets your business logic -- typed clients, explicit states, and defensive error handling make that seam reliable.



# KNOWLEDGE CHECK (1 OF 2)

*Test your understanding of Axios, HTTP methods, and data formats used when connecting React to a Spring Boot REST API.*

**1. Which library is commonly used in React to make HTTP requests to a Spring Boot API?**

- A. fetch
- B. Axios**
- C. jQuery
- D. Lodash

**3. What format is typically used to send data from React to a Spring Boot REST API?**

- A. XML
- B. JSON**
- C. YAML
- D. Plain Text

**2. Which HTTP method is typically used to create a new resource?**

- A. GET
- B. POST**
- C. PUT
- D. DELETE

**4. Which hook is commonly used in React to fetch data from an API when a component loads?**

- A. useState
- B. useEffect**
- C. useContext
- D. useRef

**KEY TAKEAWAY** Axios is the common React HTTP client; POST creates a resource; JSON is the standard payload format; useEffect is the hook that fetches data when a component loads.

# KNOWLEDGE CHECK (2 OF 2)

*One more multiple-choice question on status codes, plus a real code-recognition exercise from the curriculum worksheet.*

**5. Which status code indicates that a resource was created successfully?**

- A. 200 OK
- B. 201 Created**
- C. 204 No Content
- D. 400 Bad Request

## CODE RECOGNITION -- WHAT DOES THIS PRINT?

```
const [data, setData] = useState(null);
const [loading, setLoading] = useState(true);

useEffect(() => {
  axios.get('/api/tasks')
    .then(res => { setData(res.data); setLoading(false); })
    .catch(err => {
      console.log('Error:', err.message);
      setLoading(false);
    });
}, []);

console.log(loading, data);
```

## 6. WHAT IS THE OUTPUT ON THE FIRST RENDER?

- **A. true, null**
- B. false, []
- C. false, undefined
- D. false, list of tasks (array)

**KEY TAKEAWAY** 201 Created signals a successful POST; the bare console.log runs during the first render, before the effect's fetch resolves -- so it prints the initial state: true, null.

## MODULE 35 COMPLETE

# Frontend to API Integration

You can now connect a React frontend to a Spring Boot REST API with Axios and fetch, handle every response and failure class gracefully, and apply frontend security best practices.